

COMUNIDADE CONECTADA

Smart HAS — Fase 4

Relatorio de Atividade — Desenvolvimento Mobile com Flutter

Aluno	Yuri Sousa
RM	556163
Turma	3SIOA — 3o Ano
Ano Letivo	2026
Disciplina	Desenvolvimento de Aplicativos Mobile
Projeto	Comunidade Conectada — Smart HAS
Fase Anterior (3)	Android Nativo — Kotlin + Jetpack Compose + Room
Fase Atual (4)	Multiplataforma — Flutter 3.x + Dart + Provider

Parte 1 — Evolucoes da Fase 3 para a Fase 4

A Fase 3 entregou um aplicativo Android Nativo funcional com arquitetura robusta (Room, ViewModel, Repository, Retrofit) e integracao real com a API Google Gemini para funcionalidades de IA. A Fase 4 representa uma migracao estrategica para Flutter, mantendo toda a proposta de valor do produto e adicionando novas capacidades.

1.1 O que existia na Fase 3 (Kotlin)

- Arquitetura MVVM completa: ViewModel + StateFlow + Repository como Single Source of Truth
- Room Database com 7 entidades relacionais: User, Resource, Message, Rating, Transaction, Event, Post
- Persistencia offline-first: dados sobrevivem ao fechamento do app e sao recarregados via Flow
- LLManager conectado ao Google Gemini API para sugestao de categoria e melhoria de descricao
- ChatbotScreen com typing indicator e manutencao de contexto de conversa
- MainScreen como Shell com Bottom Navigation persistente e FAB centralizado
- seedDatabase() com 5 usuarios e 5 recursos ficticios para facilitar demonstracoes
- Integracao Retrofit para comunicacao com APIs externas

1.2 O que evoluiu na Fase 4 (Flutter)

- Migracao para Flutter 3.x: mesmo produto, agora rodando em Android e iOS a partir de um unico codebase
- Provider Pattern substituindo ViewModel/StateFlow: AuthProvider, ResourceProvider e ChatProvider
- Google Maps integrado (MapScreen) com marcadores dinamicos e localizacao do usuario — funcionalidade inedita
- Melhoria e sugestao de descricoes via OpenRouter API (comportamento equivalente ao LLManager da Fase 3)
- 10 telas implementadas, incluindo Onboarding, Chat individual, Chatbot e Detalhe de Recurso
- Firebase Cloud Messaging planejado para notificacoes push (dependencias declaradas no pubspec.yaml)
- Componentizacao com widgets reutilizaveis (ResourceCard) e separacao clara entre camadas

1.3 Tabela Comparativa Fase 3 x Fase 4

Aspecto	Fase 3 — Kotlin	Fase 4 — Flutter
Linguagem / UI	Kotlin + Jetpack Compose (Nativo Android)	Flutter 3.x + Dart (Multiplataforma)
Gerenciamento de Estado	ViewModel + StateFlow	Provider (Auth, Resource, Chat)
Persistencia Local	Room Database (offline-first, 7 entidades)	Em memoria (sem persistencia local)
Injecao de Dependencias	Service Locator manual (ComunidadeConectadaApp)	Nao aplicado formalmente

Integracao com IA	LLMManager -> Google Gemini API (real)	GeminiService -> OpenRouter API (real)
Chatbot	ChatbotScreen c/ typing indicator + contexto	ChatbotScreen c/ historico de mensagens
Navegacao	MainScreen (Shell) + NavHost + Bottom Nav	Navigator 2.0 + 10 telas definidas
Mapas	Nao implementado na Fase 3	MapScreen c/ google_maps_flutter
Push Notifications	Nao implementado na Fase 3	Firebase FCM (planejado, pubspec pronto)
Seed de Dados	seedDatabase() com 5 usuarios + 5 recursos	Dados mockados inline

Parte 2 — Arquitetura de Conectividade

A solução Comunidade Conectada opera em dois cenários: ambiente local controlado (rede Wi-Fi corporativa/doméstica) e acesso a Internet com múltiplos usuários simultâneos. A arquitetura foi projetada para ser resiliente e escalável nos dois contextos.

2.1 Topologia de Rede

Camada de dispositivos e acesso local:

- Dispositivos móveis (Android/iOS) conectados via Wi-Fi ao roteador local
- O roteador executa NAT, traduzindo os IPs privados (192.168.x.x) para o IP público ao acessar serviços externos
- Sensores e dispositivos IoT (expansão futura) conectados a mesma rede via MQTT broker local

Camada de serviços externos (Internet):

- App Flutter --> HTTPS --> OpenRouter API (IA): sugestão e melhoria de descrições, chatbot
- App Flutter --> HTTPS --> Google Maps API: mapa interativo, marcadores de recursos
- App Flutter --> HTTPS/WebSocket --> Firebase: FCM (push) e banco em tempo real
- Sensores IoT --> MQTT (TCP/UDP) --> Backend --> App Flutter

2.2 Roteamento, NAT e Protocolos de Transporte

- NAT (Network Address Translation): o roteador converte o IP privado do dispositivo para um único IP público ao estabelecer conexões com a Internet, usando PAT (Port Address Translation) para múltiplos dispositivos simultâneos
- TCP: utilizado em todas as comunicações HTTP/HTTPS com APIs REST (OpenRouter, Google Maps, Firebase), garantindo entrega ordenada e confiável dos dados
- UDP: previsto para comunicação com sensores IoT via MQTT em ambientes de baixa latência, onde pequenas perdas de pacote são toleráveis em prol da velocidade
- TLS 1.2+: camada de criptografia sobre TCP em todas as chamadas externas do app, protegendo credenciais e dados dos usuários em trânsito

2.3 Modelo Cliente-Servidor e Protocolos de Aplicação

- HTTP/REST: padrão principal entre o app Flutter (cliente) e as APIs externas (servidor). O app envia requisições POST/GET com corpo JSON e recebe respostas estruturadas. Implementado em GeminiService com tratamento de erros HTTP e timeout de 45 segundos
- MQTT (previsto): protocolo publish/subscribe de baixo overhead para comunicação com sensores IoT. Ideal para o contexto IoT do Smart HAS onde bandas limitadas e alta frequência de eventos exigem eficiência
- Firebase Realtime / Firestore (HTTPS + WebSocket): usado para FCM (entrega garantida de push notifications) e para sincronização de dados de chat em tempo real
- Google Maps SDK (gRPC interno): a biblioteca google_maps_flutter abstrai a comunicação com os servidores do Maps via gRPC, expondo uma API de alto nível para o desenvolvedor

Parte 3 — Monitoramento e Gerenciamento

3.1 Elementos da Rede e da Aplicação a Monitorar

Infra e conectividade:

- Disponibilidade das APIs externas: OpenRouter, Google Maps Platform, Firebase FCM
- Latência das requisições HTTP entre app e serviços (monitorar p50, p95, p99)
- Taxa de erros por endpoint (4xx: erros do cliente, 5xx: erros do servidor)
- Consumo de cota das APIs com limite (OpenRouter free tier, Google Maps billing)

Aplicação e dispositivos:

- Status de dispositivos IoT: online/offline, timestamp da última leitura
- Taxa de entrega de notificações push FCM (enviadas vs. entregues vs. abertas)
- Uso de memória e CPU nos dispositivos via Firebase Performance Monitoring
- Logs de autenticação: tentativas de login com falha, tokens expirados
- Integridade da base Room (Fase 3): verificar se as 7 entidades estão sendo lidas/escritas sem erros

3.2 Painel de NOC — KPIs de Operação

KPI	Meta	Descrição
Disponibilidade do App	> 99,5% uptime/mes	Percentual de tempo online do serviço
Latência HTTP (p95)	< 500ms	Tempo de resposta das chamadas REST
Taxa de Erro HTTP	< 1%	Erros 4xx/5xx sobre total de requisições
Falha nas chamadas IA	< 5%	Erros nas chamadas ao OpenRouter/Gemini
Entrega FCM (push)	< 5 segundos	Latência das notificações push
Recursos com GPS válido	100%	Recursos com lat/long preenchidos
Usuários Ativos (DAU)	Crescimento mensal	Usuários únicos por dia
Persistência Room (Fase 3)	0 dados perdidos	Dados retidos após reinicialização do app

3.3 Segurança, Escalabilidade e Resiliência

Segurança:

- Todas as comunicações externas via HTTPS com TLS 1.2 ou superior
- Autenticação via Firebase Auth com tokens JWT de curta duração e renovação automática
- Chaves de API armazenadas em variáveis de ambiente (.env) e nunca expostas no repositório
- Validação de entrada no cliente (Flutter) e no servidor para prevenir injeção e dados malformados

Escalabilidade:

- Backend Firebase escala horizontalmente de forma automática e gerenciada pelo Google

- OpenRouter e Google Maps são serviços gerenciados: sem provisionamento de infraestrutura própria
- O modelo offline-first da Fase 3 (Room) reduz a dependência de conectividade constante

Resiliência:

- Timeout configurado a 45 segundos no GeminiService, com mensagem amigável ao usuário em caso de falha
 - Modo offline parcial: Room (Fase 3) mantém dados locais; Flutter usa cache em memória enquanto app está aberto
 - Retry logic previsto para chamadas críticas (FCM, sincronização de recursos com a API)
-

Parte 4 — Implementação Mobile com Flutter

4.1 Principais Telas Implementadas

O app possui 10 telas cobrindo os fluxos principais do produto:

- OnboardingScreen — apresentação animada do conceito para novos usuários
- LoginScreen — autenticação com validação de formulário e feedback de erros
- HomeScreen — feed principal com lista de recursos disponíveis via ResourceProvider
- MapScreen — mapa interativo Google Maps com marcadores dos recursos e localização do usuário
- ChatListScreen — lista de conversas ativas, consumindo ChatProvider
- ChatScreen — conversa individual com diferenciação visual de mensagens próprias e do outro usuário
- ChatbotScreen — assistente ConectaIA com histórico de mensagens e chamadas ao OpenRouter
- ProfileScreen — dados do usuário logado, histórico de ações e logout
- OfferResourceScreen — formulário para cadastrar nova doação/serviço, com botões de IA para melhorar descrição
- ResourceDetailScreen — detalhes completos de um recurso, incluindo coordenadas e status

4.2 Boas Práticas: Widgets, Estado e Componentização

O projeto segue rigorosamente os princípios de arquitetura Flutter recomendados:

- Separação de responsabilidades: models/ (dados), providers/ (estado), screens/ (UI), services/ (APIs), widgets/ (componentes)
- Widgets stateless sempre que possível: ResourceCard, telas de detalhe recebem dados via parâmetros
- Widgets stateful apenas onde necessário: formulários, telas com animações locais
- Theme centralizado em AppTheme: cores, tipografia e estilos definidos em um único lugar, aplicados globalmente

4.3 Organização com Provider

O padrão Provider gerencia todo o estado global do app, injetado na raiz da árvore via MultiProvider no main.dart:

- AuthProvider: estado de autenticação (usuário logado, token, login/logout). Equivalente ao conceito de AuthViewModel da Fase 3
- ResourceProvider: lista de recursos, filtragem por categoria, coordenadas lat/long para o mapa. Equivalente ao ResourceViewModel + Repository da Fase 3
- ChatProvider: lista de conversas e mensagens. Equivalente ao ChatViewModel da Fase 3

Telas consomem o estado via Consumer<T>(builder: ...) para reconstruir somente o widget afetado, ou via Provider.of<T>(context, listen: false) para ações pontuais sem reconstrução. Componentes presentational (como ResourceCard) recebem dados por parâmetros, mantendo-se desacoplados do estado global.

Parte 5 — Mapas e Geolocalizacao

A MapScreen e uma das principais evolucoes da Fase 4 em relacao a Fase 3, que nao possuia integracao com mapas. Ela exhibe em tempo real os recursos cadastrados pela comunidade sobre um mapa interativo.

5.1 Implementacao

- Pacote: `google_maps_flutter`: ^2.5.3 com suporte nativo Android e iOS
- Camera inicial centrada em Sao Paulo (-23.5631, -46.6544) com zoom 12
- Marcadores dinamicos gerados a partir dos recursos no `ResourceProvider` que possuem coordenadas lat/lng validas
- `InfoWindow` ao tocar no marcador: exhibe titulo e categoria do recurso
- `BottomSheet` modal ao tap no marcador: mostra detalhes completos com opcao de abrir `ResourceDetailScreen`
- `myLocationEnabled`: true — exhibe ponto azul da localizacao atual do usuario
- `BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueAzure)` — marcadores azuis diferenciam pontos de recurso da localizacao do usuario

5.2 Potencial de Expansao

- Filtro por categoria diretamente no mapa (ex.: exhibir apenas 'Alimentos' ou 'Servicos')
- Raio de busca ajustavel pelo usuario para encontrar recursos proximos
- Roteamento ate o ponto de entrega integrado ao Google Maps Directions API
- Integracao com dados de sensores IoT para exhibir status de dispositivos no mapa

Parte 6 — Integracao com Firebase e Web Services

6.1 Servicos Web Integrados

Servico 1 — OpenRouter API (Inteligencia Artificial):

- Endpoint: `https://openrouter.ai/api/v1/chat/completions`
- Modelo: `baidu/cobuddy:free` (tier gratuito via OpenRouter)
- Funcionalidade 1: chatbot ConectaIA (ChatbotScreen) com historico de conversa
- Funcionalidade 2: melhoria automatica de descricao na OfferResourceScreen (equivalente ao `LLMManager.improveDescription()` da Fase 3)
- Funcionalidade 3: sugestao de categoria com base no texto digitado (equivalente ao `LLMManager.suggestCategories()` da Fase 3)
- Implementacao: GeminiService com timeout 45s, tratamento de erros HTTP e rate limiting

Servico 2 — Google Maps Platform:

- SDK: `google_maps_flutter ^2.5.3` integrado nativamente para Android e iOS
- Maps SDK for Android/iOS: renderizacao do mapa, marcadores, camera e gestos
- Localizacao do usuario em tempo real exibida no mapa
- Coordenadas lat/lng dos recursos persistidas junto ao modelo ResourceEntity

6.2 Firebase Cloud Messaging (FCM)

Fluxo de notificacao push planejado e documentado para implementacao em producao:

- Trigger: usuario cadastra novo recurso em categoria de interesse de outro usuario
- Backend envia mensagem FCM ao device token do usuario interessado
- Payload: `{ title: 'Nova doacao disponível!', body: 'Um recurso de [categoria] foi cadastrado perto de voce.' }`
- Comportamento: ao tocar na notificacao, o app abre diretamente a MapScreen com foco no marcador do novo recurso
- Status tecnico: dependencias `firebase_core` e `firebase_messaging` declaradas no `pubspec.yaml`; ativacao requer `google-services.json` e configuracao do projeto Firebase

Parte 7 — Comparacao Flutter (Fase 4) vs Kotlin (Fase 3)

Esta comparacao e baseada na experiencia real de desenvolver o mesmo produto — Comunidade Conectada — nas duas plataformas ao longo das duas fases do projeto.

7.1 Produtividade e Velocidade de Desenvolvimento

Flutter foi significativamente mais produtivo para este projeto. O hot reload permitiu iterar na UI em segundos, enquanto o Kotlin exigia recompilacao a cada alteracao. A construcao das 10 telas em Flutter levou menos tempo do que construir as mesmas telas em Kotlin/Compose, mesmo com a curva de aprendizado do Dart.

7.2 Flutter — Pontos Positivos

- Hot reload: alteracoes de UI refletem instantaneamente sem perder o estado da tela — maior diferencial de produtividade
- Multiplataforma real: o mesmo codebase roda em Android e iOS. Na Fase 3 (Kotlin), era necessario um projeto separado para iOS
- Dart tipado e conciso: sintaxe mais enxuta para declarar widgets, menos boilerplate que XML + Activity + ViewModel
- Material Design 3 nativo: componentes visuais modernos ja prontos e altamente customizaveis
- Provider mais simples que ViewModel + StateFlow + LiveData para casos de uso padrao
- pub.dev: ecossistema vasto com pacotes para Maps, camera, push notifications, imagens

7.3 Flutter — Pontos Negativos

- Sem persistencia local nativa equivalente ao Room: o pacote sqflite existe, mas e mais verboso e sem suporte a migrations robustas como o Room
- APK maior: o runtime Flutter e embutido no APK, resultando em arquivos maiores que apps Kotlin nativos
- Acesso a hardware especifico: funcionalidades como Bluetooth, NFC ou sensores avancados exigem platform channels e codigo nativo adicional
- Arvores de widgets profundas: a filosofia 'tudo e widget' pode tornar o codigo verbose em telas complexas

7.4 Kotlin — Pontos Positivos

- Room Database: ORM nativo com suporte a migrations, relacionamentos e reactive queries via Flow — muito superior ao sqflite do Flutter
- Integracao perfeita com o ecossistema Android: Jetpack WorkManager, Paging 3, DataStore, Biometric
- Kotlin Coroutines: sistema de concorrencia elegante e poderoso, especialmente com viewModelScope e Flow
- APK menor e melhor performance em dispositivos de entrada
- Acesso total ao hardware sem platform channels: camera, Bluetooth, NFC, sensores

7.5 Kotlin — Pontos Negativos

- Apenas Android: o codigo nao e reutilizavel para iOS ou Web, aumentando o custo de desenvolvimento e manutencao para multi-plataforma

- Mais verboso para UIs: mesmo com Jetpack Compose, a quantidade de boilerplate (ViewModel + Factory + StateFlow + Composable) é maior que o equivalente Flutter
- Sem hot reload: a inexistência de hot reload torna o ciclo de desenvolvimento de UI mais lento
- Curva do Jetpack Compose: ainda mais jovem que o Flutter, com mudanças de API frequentes

7.6 Conclusão Comparativa

Para o perfil do Comunidade Conectada — app social com mapa, chat, IA e necessidade de rodar em iOS e Android — Flutter foi a escolha técnica mais acertada para a Fase 4. A produtividade foi maior, o visual ficou equivalente ao Kotlin com menos esforço, e a multiplataforma é um diferencial real.

Por outro lado, a Fase 3 em Kotlin produziu uma base de dados muito mais sólida (Room com 7 entidades, offline-first) que o Flutter ainda não replicou completamente. Em produção, o ideal seria usar os pontos fortes de ambos: Flutter para a UI multiplataforma e Room (via FFI ou Kotlin Multiplatform) para a camada de persistência.